

Here is the companion **Docker Compose Cheatsheet** to help you define, run, and manage multi-container Docker applications.

1. What is Docker Compose?

While a Dockerfile defines how to build a **single image**, a docker-compose.yml file defines how **multiple containers** (services, networks, and volumes) interact with each other.

2. Anatomy of a docker-compose.yml File

Here is a modern, production-ready template setting up a Node.js web application connected to a PostgreSQL database.

YAML

```
# The "version" field is deprecated in modern Compose; just start with "services"
services:

  # 1. Our Frontend/Backend App Service
  web:
    build:
      context: .          # Look in the current directory
      dockerfile: Dockerfile # Use this Dockerfile
    ports:
      - "8080:8080"      # Map host port 8080 to container port 8080
    environment:
      - NODE_ENV=development
      - DB_HOST=db        # Uses the service name "db" as the hostname
      - DB_PASSWORD_FILE=/run/secrets/db_password
    volumes:
      - ../usr/src/app    # Bind mount for hot-reloading (local dev only)
      - /usr/src/app/node_modules # Prevent host node_modules from overwriting container
    depends_on:
      db:
        condition: service_healthy # Wait for the database to be healthy before starting
    secrets:
      - db_password
```

```
# 2. Database Service
db:
  image: postgres:16-alpine # Use a pre-built official image
  restart: always # Restart container if it crashes or the daemon restarts
  environment:
    POSTGRES_USER: admin
    POSTGRES_PASSWORD_FILE: /run/secrets/db_password
    POSTGRES_DB: my_database
  volumes:
    - db_data:/var/lib/postgresql/data # Persist data even if container is deleted
  ports:
    - "5432:5432"
  healthcheck: # Ensures other services only connect when DB is ready
    test: ["CMD-SHELL", "pg_isready -U admin -d my_database"]
    interval: 10s
    timeout: 5s
    retries: 5
  secrets:
    - db_password
```

3. Define persistent storage

```
volumes:
  db_data: # Named volume for postgres data
```

4. Define sensitive data (Secrets)

```
secrets:
  db_password:
    file: ./db_password.txt # Keeps passwords out of your Git repository
```

3. Essential Docker Compose Commands

Always run these commands from the directory containing your docker-compose.yml file.

Command	Purpose
docker compose up	Builds, (re)creates, starts, and attaches to containers.

docker compose up -d	Starts containers in detached mode (runs in the background).
docker compose down	Stops and removes containers, networks, and images created by up.
docker compose down -v	Stops containers and deletes persistent volumes (wipes databases clean).
docker compose ps	Lists the status of all containers managed by this Compose file.
docker compose logs -f	Follow log output from all running services.
docker compose logs -f <service>	Follow logs for a <i>specific</i> service (e.g., docker compose logs -f web).
docker compose exec <service> <cmd>	Runs a command inside a running container (e.g., docker compose exec db psql).
docker compose build	Forces a rebuild of any images specified with the build: key.

4. Key Concepts Explained

Volumes: Named vs. Bind Mounts

- **Bind Mounts (./relative/path:/app):** Maps a directory on your host machine to the container. Excellent for **local development** because code changes instantly sync to the container without rebuilding the image.

- **Named Volumes (db_data:/var/lib/...):** Managed completely by Docker. Best for **production and database storage** because they are high-performance and isolated from host-side system issues.

Networking (Automatic)

You don't need to manually configure IP addresses or bridge networks.

- Docker Compose automatically creates a single, private network for your services.
- Every container can reach any other container simply by using its **service name** as the hostname. (e.g., inside the web container, you connect to the database via `mongodb://db:27017` or `postgres://db:5432`).

Environment Variables (.env)

Compose automatically looks for a .env file in the same directory. You can use this to keep credentials and configurations out of your main YAML file.

.env file:

Plaintext

```
PORT=8080
DB_USER=superadmin
```

docker-compose.yml file:

YAML

```
services:
  web:
    image: my-app
    ports:
      - "${PORT}:${PORT}"
```

